Ejercicio 1: Implementar una pila de enteros

Prompt redactado:

Estructura solicitada: Clase PilaEnteros en Java que utilice internamente un arreglo estático de tamaño fijo.

Manejo de arreglo e índice top: Explica que el índice top rastrea la posición del elemento superior. Debe inicializarse en -1 para indicar que la pila está vacía. Al hacer push, top se incrementa en 1 antes de insertar; al hacer pop, se extrae el valor y top se decrementa en 1.

Operaciones a implementar: push(int x), pop(), peek(), isEmpty(), isFull() y size().

Casos límite:

- Si se intenta hacer push cuando isFull() es verdadero (top == capacidad - 1), debe lanzar una excepción o mostrar mensaje de desbordamiento (Stack Overflow).
- Si se intenta hacer pop o peek cuando isEmpty() es verdadero (top == -1), debe lanzar excepción o mensaje de subdesbordamiento (Stack Underflow).

Código y Pruebas: Implementa la clase y un método main probando inserción hasta llenar la pila y extracción hasta vaciarla, con comentarios en cada método.

Ejercicio 2: Simulador de torre de platos

Prompt redactado:

Estructura solicitada: Programa simulador de una torre de platos utilizando el concepto de Pila.

Justificación teórica: Explica en los comentarios por qué este problema se resuelve estrictamente con una Pila (LIFO) y no con una Cola: el último plato que se coloca sobre la torre es necesariamente el primero en retirarse para no romper la pila.

Operaciones del programa:

1. agregarPlato(String plato): Apila un plato por nombre o número.
2. retirarPlato(): Desapila y devuelve el plato superior.
3. consultarPlatoSuperior(): Retorna el plato de arriba sin quitarlo (peek).

Código y Pruebas: Muestra la simulación agregando 3 platos, consultando el superior y retirándolos uno a uno por consola.

Ejercicio 3: Validar paréntesis balanceados

Prompt redactado:

Estructura solicitada: Algoritmo en Java para verificar si una expresión matemática tiene los paréntesis balanceados mediante una Pila de caracteres.

Funcionamiento con Pila:

- Recorrer la cadena carácter por carácter.
- Al encontrar un paréntesis de apertura '(', realizar un push a la pila.
- Al encontrar un paréntesis de cierre ')', realizar un pop de la pila.

Casos límite de invalidez:

1. Si se encuentra un ')' y la pila está vacía al intentar hacer pop.
2. Si se termina de recorrer la cadena y la pila NO está vacía (quedaron paréntesis abiertos sin cerrar).

Código y Pruebas: Implementa una función boolean esBalanceado(String expresion) y pruébala con:

- "(5 + 3) * (2 + 1)" (debe devolver Válido).
- "(5 + 3)) * (2 + 1" (debe devolver Inválido).

Ejercicio 4: Historial de navegación

Prompt redactado:

Estructura solicitada: Simulador de historial de navegación web en Java utilizando una Pila.

Justificación LIFO: Explica en los comentarios por qué el historial funciona bajo el principio LIFO (Last In, First Out): la última página web visitada es la primera a la que volvemos cuando presionamos el botón "Atrás" del navegador.

Funcionalidades:

1. visitarPagina(String url): Apila la URL ingresada.
2. volverAtras(): Realiza un pop para regresar a la URL anterior e informarla por pantalla.
3. paginaActual(): Muestra la URL donde se encuentra posicionado el usuario.

Código y Pruebas: Simula la visita a 3 sitios web distintos y realiza dos llamadas a volverAtras(), mostrando el estado del historial en consola.

Ejercicio 5: Implementar una pila genérica

Prompt redactado:

Estructura solicitada: Clase genérica Pila<T> en Java utilizando arreglos.

Justificación de Genéricos: Explica qué problema resuelve el uso de Generics (<T>) frente a una pila fija tipo int: permite reutilizar la misma estructura para almacenar cualquier tipo de objeto (Integer, String, objetos propios) garantizando la seguridad de tipos en tiempo de compilación sin duplicar código.

Manejo interno: Crear el arreglo genérico mediante casting explícito (T[]) new Object[capacidad] debido a las restricciones de reificación de Java.

Operaciones: push, pop, peek, isEmpty, isFull y size.

Código y Pruebas: Instanciar un objeto Pila<String> para textos y un objeto Pila<Integer> para enteros, probando sus operaciones.

Ejercicio 6: Implementar una cola simple de enteros

Prompt redactado:

Estructura solicitada: Clase ColaEnteros en Java basada en un arreglo de tamaño fijo.

Manejo de índices front y rear:

- Explica que front indica la posición del primer elemento a salir y rear indica la posición del último elemento ingresado.
- Inicializar front = 0, rear = -1 y una variable contador de elementos.
- Al ejecutar enqueue, rear incrementa en 1 y se guarda el dato; al ejecutar dequeue, se extrae el dato en front y front incrementa en 1.

Operaciones: enqueue(int x), dequeue(), front(), isEmpty(), isFull() y size().

Código y Pruebas: Implementar la clase con validación de desbordamiento/subdesbordamiento y un método main de prueba.

Ejercicio 7: Sistema de turnos para atención

Prompt redactado:

Estructura solicitada: Programa simulador de sistema de turnos de atención para una oficina mediante una Cola.

Justificación FIFO: Explica en el código por qué se utiliza la disciplina FIFO (First In, First Out) y no LIFO: se debe respetar el orden estricto de llegada para ser justos en la atención al cliente (el primero en llegar es el primero en ser atendido).

Operaciones:

1. agregarPersona(String nombre): Encola una persona al final de la fila (enqueue).

2. atenderSiguiente(): Desencola y muestra quién está siendo atendido (dequeue).
3. consultarPrimero(): Muestra quién es el próximo a ser atendido sin sacarlo de la cola (front).

Código y Pruebas: Encolar 4 clientes, consultar quién está primero y atender a los 2 primeros.

Ejercicio 8: Cola de impresión

Prompt redactado:

Estructura solicitada: Sistema simulador de cola de impresión de documentos en Java.

Modelado: Crear una clase Documento con atributos nombre (String) y paginas (int).

Justificación FIFO: Explica cómo la estructura de Cola representa fielmente la secuencia lógica de un Spooler de impresión, garantizando que los trabajos se procesen estrictamente en el orden cronológico en que fueron enviados.

Operaciones: agregarDocumento(Documento doc), imprimirSiguiente() y mostrarCola().

Código y Pruebas: Encolar 3 documentos y procesar la impresión en orden FIFO mostrando los detalles de cada trabajo por pantalla.

Ejercicio 9: Problema de desperdicio de espacio en cola simple

Prompt redactado:

Estructura solicitada: Programa en Java que demuestre de forma explícita el límite estructural y el desperdicio de espacio de una cola lineal o simple sobre arreglo.

Demostración requerida:

1. Crear un arreglo de tamaño fijo N (ej. $\mathrm{N} = 5$) y llenarlo mediante enqueue.
2. Realizar varias operaciones de dequeue para liberar las primeras posiciones.
3. Intentar realizar un nuevo enqueue y mostrar cómo el sistema indica que la cola está llena (rear == N - 1), a pesar de tener casilleros vacíos e inutilizables al principio del arreglo.

Código y Pruebas: Imprimir por consola el estado interno del arreglo y los valores de front y rear tras cada operación para evidenciar el desperdicio de posiciones.

Ejercicio 10: Cola circular

Prompt redactado:

Estructura solicitada: Clase ColaCircular en Java sobre arreglo de tamaño fijo que resuelva el problema del desperdicio de memoria.

Manejo del operador Módulo (%): Explica detalladamente cómo el operador módulo permite que los índices front y rear "den la vuelta" al llegar al final del arreglo, conectando la última posición con la primera:

- Avanzar rear: rear = (rear + 1) % capacidad.
- Avanzar front: front = (front + 1) % capacidad.

Detección de estados: Explicar cómo llevar la cuenta con un atributo count para distinguir fácilmente cuándo la cola está vacía (count == 0) o llena (count == capacidad).

Código y Pruebas: Demostrar que se pueden seguir insertando elementos tras hacer varios dequeue, reutilizando con éxito los espacios liberados al inicio.